

Experiment 'p_4_2_a_min_max_seq' Results

June 5, 2026

Experiment outcome: SUCCESS

Bad responses: 0

Responses containing assume : 0

Responses containing expect : 0

Resolution attempts: 1

Hard fails (resolution): 0

Soft fails (resolution): 0

Verification attempts: 1

Problem Specification

Problem name: p_4_2_a_min_max_seq

Natural language statement: null

Method signature: p_4_2_a_min_max_seq(inputs: seq<int>) returns (min: int, max: int)

Ensures

- forall i :: 0 <= i < |inputs| ==> min <= inputs[i]
- forall i :: 0 <= i < |inputs| ==> max >= inputs[i]
- exists i :: 0 <= i < |inputs| && min == inputs[i]
- exists i :: 0 <= i < |inputs| && max == inputs[i]

Requires

- 0 < |inputs|

GenAI interactions

Below you will find all interactions between the ‘user’ (program) and the ‘assistant’ (OpenAI).

Program → GenAI

You are given the following task to perform in Dafny:

Write a method to satisfy the below requirements.

The signature should be:

`p_4_2_a_min_max_seq(inputs: seq<int>) returns (min: int, max: int)`

The method should respect the following contract:

requires `0 < |inputs|`, ensures forall `i :: 0 <= i < |inputs| ==> min <= inputs[i]`,
 ↪ ensures forall `i :: 0 <= i < |inputs| ==> max >= inputs[i]`, ensures exists `i :: 0`
 ↪ `<= i < |inputs| && min == inputs[i]`, ensures exists `i :: 0 <= i < |inputs| && max`
 ↪ `== inputs[i]`

Produce and show only the Dafny body of this method, including the curly braces that
 ↪ surround it. Do not show the signature nor contract. You must not use 'assume' nor
 ↪ 'expect' anywhere in your code.

GenAI → Program

System fingerprint: null

ID: chatcmpl-DnLHi7ycIFUEY4As2PRZ5MuRpNU4z

Model: gpt-5.5-2026-04-23

Created at: 1780652262

Finish reason: stop

Usage: 402 tokens in, and 751 tokens out

```
{
  min := inputs[0];
  max := inputs[0];
  ghost var minIdx := 0;
  ghost var maxIdx := 0;
  var j := 1;

  while j < |inputs|
    invariant 1 <= j <= |inputs|
    invariant 0 <= minIdx < j
    invariant 0 <= maxIdx < j
    invariant min == inputs[minIdx]
    invariant max == inputs[maxIdx]
    invariant forall i :: 0 <= i < j ==> min <= inputs[i]
    invariant forall i :: 0 <= i < j ==> max >= inputs[i]
    decreases |inputs| - j
  {
    if inputs[j] < min {
```

```

    min := inputs[j];
    minIdx := j;
  }

  if inputs[j] > max {
    max := inputs[j];
    maxIdx := j;
  }

  j := j + 1;
}

```

Final Program

```

method p_4_2_a_min_max_seq(inputs: seq<int>) returns (min: int, max: int)
  requires 0 < |inputs|
  ensures forall i :: 0 <= i < |inputs| ==> min <= inputs[i]
  ensures forall i :: 0 <= i < |inputs| ==> max >= inputs[i]
  ensures exists i :: 0 <= i < |inputs| && min == inputs[i]
  ensures exists i :: 0 <= i < |inputs| && max == inputs[i]
{
  min := inputs[0];
  max := inputs[0];
  ghost var minIdx := 0;
  ghost var maxIdx := 0;
  var j := 1;

  while j < |inputs|
    invariant 1 <= j <= |inputs|
    invariant 0 <= minIdx < j
    invariant 0 <= maxIdx < j
    invariant min == inputs[minIdx]
    invariant max == inputs[maxIdx]
    invariant forall i :: 0 <= i < j ==> min <= inputs[i]
    invariant forall i :: 0 <= i < j ==> max >= inputs[i]
    decreases |inputs| - j
  {
    if inputs[j] < min {
      min := inputs[j];
      minIdx := j;
    }

    if inputs[j] > max {
      max := inputs[j];
      maxIdx := j;
    }

    j := j + 1;
  }
}

```

}

Total Token Usage

Input tokens: 402

Output tokens: 751

Reasoning tokens: 512

Sum of ‘total tokens’: 1153

Experiment Timings

Overall Experiment started at 1780652261776, ended at 1780652286200, lasting 24424ms (24.42 seconds)

Iteration #1 started at 1780652261794, ended at 1780652286200, lasting 24406ms (24.41 seconds)